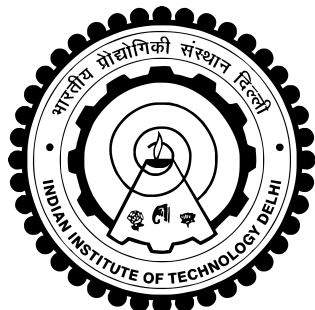


INDIAN INSTITUTE OF TECHNOLOGY DELHI

New Delhi – 110016



GIPEDI INTERNSHIP REPORT

Submitted in fulfillment of the requirements for the Internship Program

Name: Karuna Sharma

Internship Period: 15 May 2026 – 15 July 2026

Organization: Department of Electrical Engineering
Indian Institute of Technology Delhi

Supervisor: Prof. Subrat Kar

Mentor: Utkarsh Roy

July 2026

Certificate

This is to certify that Ms. **Karuna Sharma** has successfully completed her internship work titled:

”GIPEDI INTERNSHIP REPORT”

during the period from **15 May 2026 to 15 July 2026** under the guidance of **Prof. Subrat Kar**, Department of Electrical Engineering, Indian Institute of Technology Delhi.

During the internship period, she worked sincerely and diligently on various experiments and projects related to embedded systems, electronics, and web application development. The work presented in this report is a record of the activities carried out during the internship period.

Prof. Subrat Kar
Department of Electrical Engineering
Indian Institute of Technology Delhi

Date: July 15, 2026

Acknowledgement

I express my sincere gratitude to **Prof. Subrat Kar**, Department of Electrical Engineering, Indian Institute of Technology Delhi, for providing me with the opportunity to undertake this internship and for his valuable guidance and encouragement throughout the internship period.

I would also like to thank **Mr. Utkarsh Roy** for his continuous support, technical guidance, and valuable suggestions during the course of this work.

I am grateful to the faculty members, staff, and colleagues of the Department of Electrical Engineering, IIT Delhi, for providing the necessary facilities and a conducive environment for learning and experimentation.

Finally, I would like to thank everyone who directly or indirectly contributed to the successful completion of this internship.

Karuna Sharma

Contents

Certificate	1
Acknowledgement	2
1 Arduino Basic Program	6
1.1 Objective	6
1.2 Tools Required	6
1.3 Circuit Diagram	6
1.4 Arduino Code	7
1.5 Method	7
1.5.1 Learning Outcome	7
1.5.2 Problems Faced	7
1.5.3 Future Scope	7
2 Experiment 1	8
2.1 Title	8
2.2 Objective	8
2.3 Tools Required	8
2.4 Theory	8
2.5 Circuit Diagram	9
2.5.1 Arduino Code	9
2.6 Procedure	9
2.7 Observations	9
2.8 Learning Outcome	9
2.9 Problems Faced	10
2.10 Future Scope	10
3 Experiment 2	11
3.1 Title	11
3.2 Objective	11
3.3 Tools Required	11
3.4 Theory	11
3.5 Circuit Diagram	12
3.6 Arduino Code	12
3.7 Procedure	12
3.8 Observations	12
3.9 Learning Outcome	12
3.10 Problems Faced	13
3.11 Future Scope	13

4	Project on 555 Timer IC	14
4.1	Title	14
4.2	Aim	14
4.3	Objectives	14
4.4	Components Required	14
4.5	Circuit Diagram	15
4.6	Theory	15
4.7	Procedure	15
4.8	Observations	16
4.9	Applications	16
4.10	Advantages	16
4.11	Conclusion	16
5	Inverting and Non-Inverting Amplifier using Operational Amplifier	17
5.1	Aim	17
5.2	Objectives	17
5.3	Components Required	17
5.4	Theory	18
5.5	Circuit Diagrams	18
5.6	Procedure	19
5.7	Observations	19
5.8	Applications	19
5.9	Advantages	20
5.10	Conclusion	20
6	Typing Progress Report	21
6.1	Objective	21
6.2	Typing Performance	21
6.3	Typing Progress Graph	22
6.4	Observation	22
6.5	Conclusion	22
7	HTML with MySQL Database	23
7.1	Aim	23
7.2	Objectives	23
7.3	Software Required	23
7.4	Hardware Requirements	23
7.5	Introduction	24
7.6	Theory	24
7.7	Flow of the System	24
7.8	SQL Database	25
7.9	HTML Source Code	25
7.10	Advantages	25
7.11	Applications	25
7.12	Output	25
7.13	Conclusion	26

8	Dashboard Button Interface	27
8.1	Aim	27
8.2	Objective	27
8.3	Theory	27
8.4	Software Required	27
8.5	Procedure	28
8.6	Program Code	28
8.7	Screenshot	28
8.8	Observations	28
8.9	Result	29
8.10	Conclusion	29

Chapter 1

Arduino Basic Program

1.1 Objective

- Learn the components of an Arduino Uno circuit.
- Understand how to connect basic electronic components.
- Write and upload a simple Arduino program.
- Verify the circuit through simulation.

1.2 Tools Required

1. Arduino UNO
2. LED
3. Arduino IDE

1.3 Circuit Diagram

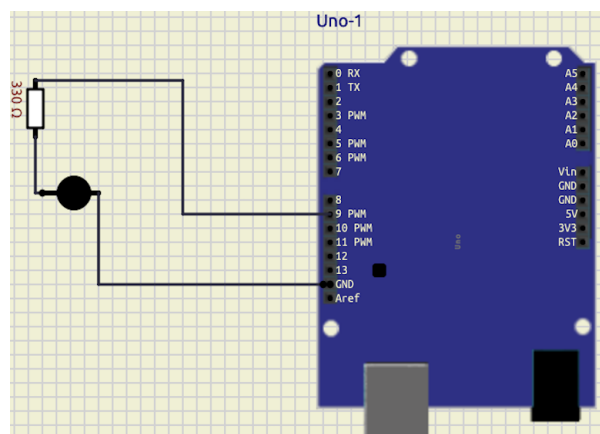


Figure 1.1: Arduino LED Blinking Circuit

1.4 Arduino Code

The complete implementation code is available in the GitHub repository referenced in the footer.

1.5 Method

The program repeatedly turns the LED ON for one second and OFF for one second, producing a continuous blinking effect.

1.5.1 Learning Outcome

- Learned how to connect an LED with Arduino Uno.
- Learned to write and upload an Arduino program.
- Understood basic digital output control.

1.5.2 Problems Faced

Problem

The LED did not blink after uploading the program.

Solution

Checked the wiring connections and uploaded the program again.

1.5.3 Future Scope

- Control multiple LEDs.
- Interface sensors with Arduino.
- Develop simple automation projects.

Chapter 2

Experiment 1

2.1 Title

Interface a Seven Segment LED Display Using a Push Button

2.2 Objective

To interface a seven-segment display with Arduino and use a push button to implement a counter.

2.3 Tools Required

1. Arduino UNO
2. Seven Segment Display (Common Cathode)
3. Push Button
4. Arduino IDE

2.4 Theory

A seven-segment display consists of seven LEDs arranged in the shape of the number 8. By turning individual LEDs ON and OFF, digits from 0 to 9 can be displayed. A push button acts as the input device that increments the displayed count every time it is pressed.

2.5 Circuit Diagram

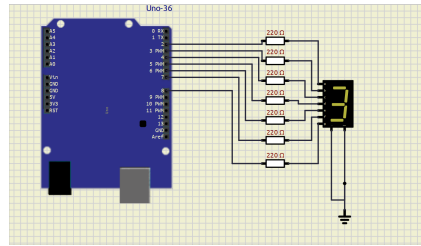


Figure 2.1: Interfacing Seven Segment Display with Arduino

2.5.1 Arduino Code

The complete implementation code is available in the GitHub repository referenced in the footer.

2.6 Procedure

1. Connect the seven-segment display to the Arduino digital pins.
2. Connect the push button to the Arduino input pin.
3. Upload the program using the Arduino IDE.
4. Press the push button repeatedly.
5. Observe the number displayed on the seven-segment display.

2.7 Observations

- The display initially shows 0.
- Each button press increments the displayed number.
- After reaching 9, the counter resets to 0.

2.8 Learning Outcome

- Learned to interface a seven-segment display with Arduino.
- Digital input was detected using a push button.
- Implemented a simple counter using Arduino.

2.9 Problems Faced

Problem

Incorrect digits were displayed on the seven-segment display due to incorrect wiring.

Solution

The segment pin connections were checked and corrected according to the circuit diagram, after which the display operated correctly.

2.10 Future Scope

- Four-digit digital counters.
- Visitor counting systems.
- Digital scoreboards.
- Electronic voting and counting systems.

Chapter 3

Experiment 2

3.1 Title

Display Different Patterns on an LED Matrix

3.2 Objective

To interface an LED matrix with Arduino Uno and display different patterns, symbols, and designs.

3.3 Tools Required

1. Arduino Uno
2. 8×8 LED Matrix
3. Arduino IDE

3.4 Theory

An LED matrix is a collection of LEDs arranged in rows and columns. By controlling each LED individually, different symbols, letters, numbers, patterns, and animations can be displayed. Arduino sends binary data to illuminate selected LEDs and generate the desired output.

3.5 Circuit Diagram

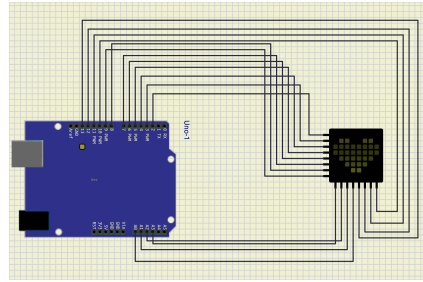


Figure 3.1: Interfacing an LED Matrix with Arduino Uno

3.6 Arduino Code

The complete implementation code is available in the GitHub repository referenced in the footer.

3.7 Procedure

1. Connect the LED matrix to the Arduino.
2. Upload the program using the Arduino IDE.
3. Run the circuit.
4. Observe the displayed pattern.
5. Modify the binary data to display different symbols and designs.

3.8 Observations

- The LED matrix successfully displayed the programmed pattern.
- Different patterns were obtained by changing the binary values.
- Letters, numbers, symbols, and simple animations can be displayed.

3.9 Learning Outcome

- Learned how to interface an LED matrix with Arduino.
- Understood row and column addressing.
- Gained experience in creating visual patterns using programming.

3.10 Problems Faced

Problem

The LED matrix did not display the correct pattern.

Solution

Checked all wiring connections and verified the binary pattern data in the program.

3.11 Future Scope

- Scrolling text displays.
- Animation systems.
- LED notice boards.
- Electronic display systems.

Chapter 4

Project on 555 Timer IC

4.1 Title

Design and Implementation of a 555 Timer Circuit

4.2 Aim

To study the working of the 555 Timer IC and generate timing pulses using an astable multivibrator circuit.

4.3 Objectives

1. To understand the operation of the 555 Timer IC.
2. To generate a continuous square-wave output.
3. To observe the charging and discharging characteristics of a capacitor.

4.4 Components Required

1. NE555 Timer IC
2. Resistor R1 (1 k Ω)
3. Resistor R2 (10 k Ω)
4. Capacitor C1 (10 μ F)
5. Breadboard
6. Connecting wires
7. 5 V DC Power Supply
8. LED

4.5 Circuit Diagram

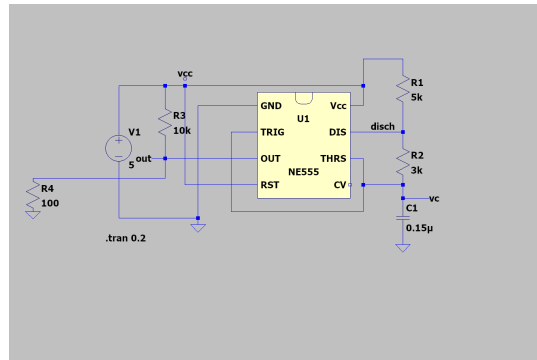


Figure 4.1: Astable Multivibrator using NE555 Timer IC

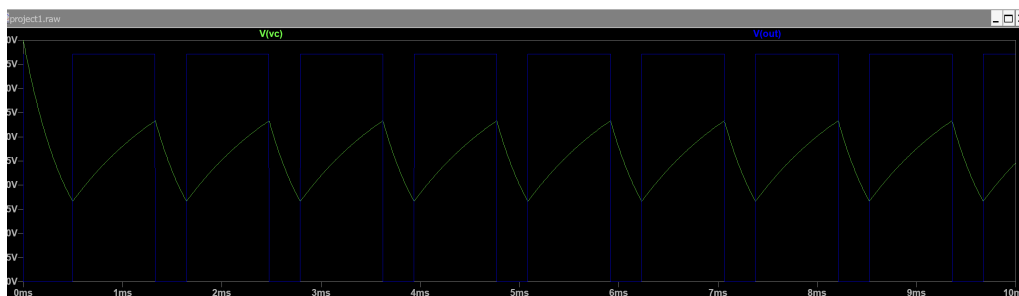


Figure 4.2: Output Waveform

4.6 Theory

The NE555 Timer is one of the most widely used integrated circuits in electronic applications. It can operate in three modes:

- Astable Mode
- Monostable Mode
- Bistable Mode

In this experiment, the IC operates in astable mode, where the output continuously switches between HIGH and LOW without requiring an external trigger.

The charging and discharging of capacitor C1 through resistors R1 and R2 determines the output frequency.

4.7 Procedure

1. Place the NE555 IC on the breadboard.
2. Connect the circuit according to the circuit diagram.

-
3. Connect the LED at the output pin through a resistor.
 4. Apply a 5 V power supply.
 5. Observe the blinking LED.
 6. Measure the output waveform if an oscilloscope is available.

4.8 Observations

- The LED blinked continuously.
- The blinking frequency depended upon the resistor and capacitor values.
- A square-wave output was obtained at the output pin.
- Changing capacitor values changed the blinking speed.

4.9 Applications

- LED Flashers
- Pulse Generators
- Clock Circuits
- Alarm Systems
- Tone Generators
- PWM Circuits

4.10 Advantages

- Low cost
- Simple circuit
- Highly reliable
- Easy to design
- Wide operating voltage range

4.11 Conclusion

The project successfully demonstrated the working principle of the NE555 Timer IC in astable mode. The circuit generated periodic square-wave pulses, illustrating how resistor and capacitor values determine the output frequency.

Chapter 5

Inverting and Non-Inverting Amplifier using Operational Amplifier

5.1 Aim

To study and analyze the operation of inverting and non-inverting amplifiers using an Operational Amplifier.

5.2 Objectives

1. To understand the working principle of inverting and non-inverting amplifiers.
2. To calculate and verify the voltage gain.
3. To compare their characteristics.
4. To study practical applications.

5.3 Components Required

1. IC 741 Operational Amplifier
2. Breadboard
3. DC Power Supply
4. Resistors
5. Connecting Wires
6. Function Generator
7. Digital Multimeter
8. Oscilloscope

5.4 Theory

An Operational Amplifier (Op-Amp) is a high-gain differential amplifier used in analog electronic circuits.

An inverting amplifier produces an output that is 180° out of phase with the input signal.

A non-inverting amplifier produces an amplified output while maintaining the same phase as the input signal.

The voltage gain of an inverting amplifier is

$$A_v = -\frac{R_f}{R_{in}}$$

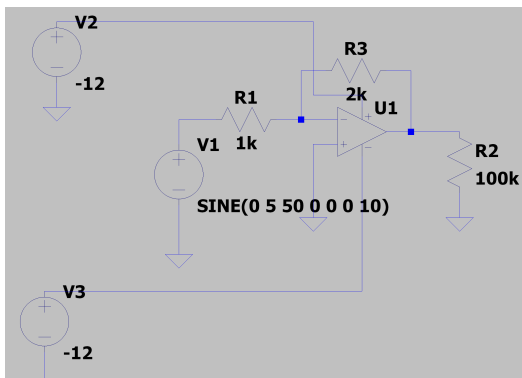
where

- R_f = Feedback Resistance
- R_{in} = Input Resistance

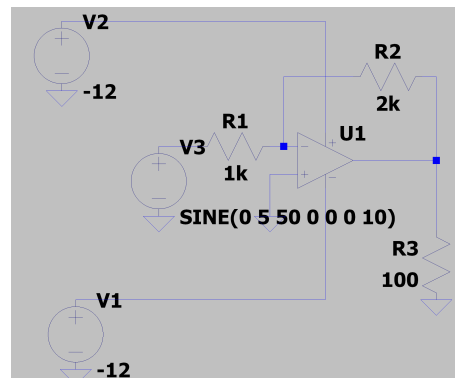
The voltage gain of a non-inverting amplifier is

$$A_v = 1 + \frac{R_f}{R_1}$$

5.5 Circuit Diagrams



(a) Non-Inverting Amplifier



(b) Inverting Amplifier

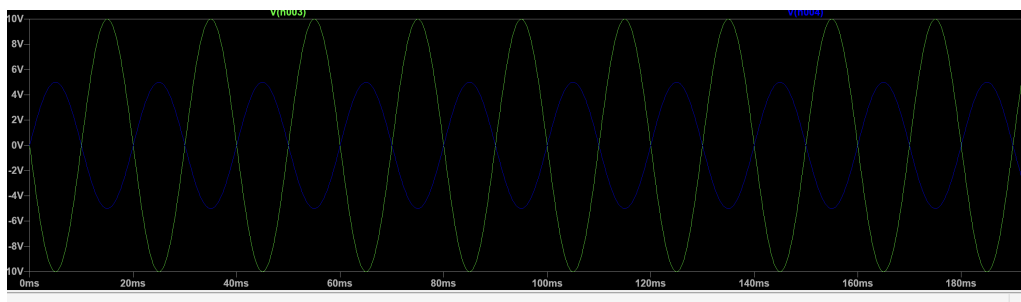


Figure 5.2: Output Waveform

5.6 Procedure

1. Assemble the inverting amplifier circuit.
2. Apply a sinusoidal input signal.
3. Observe the output waveform.
4. Measure the voltage gain.
5. Repeat the same procedure for the non-inverting amplifier.
6. Compare the results.

5.7 Observations

- The inverting amplifier output was 180° out of phase.
- The non-inverting amplifier output remained in phase.
- The measured gain closely matched the theoretical value.
- The output remained stable throughout the experiment.

5.8 Applications

Inverting Amplifier

- Signal Conditioning
- Audio Amplifiers
- Active Filters
- Analog Computing

Non-Inverting Amplifier

- Voltage Followers
- Sensor Signal Amplification
- Instrumentation Circuits
- Buffer Amplifiers

5.9 Advantages

- High input impedance
- Stable voltage gain
- Excellent linearity
- Low distortion
- Easy implementation

5.10 Conclusion

The experiment verified the working of both inverting and non-inverting Operational Amplifier configurations. The measured voltage gain closely matched theoretical calculations, demonstrating the effectiveness of Operational Amplifiers in signal amplification.

Chapter 6

Typing Progress Report

6.1 Objective

To improve typing speed and accuracy through daily practice and monitor the progress throughout the internship period.

6.2 Typing Performance

Table 6.1: Typing Progress Report

S. No.	Date	WPM	S. No.	Date	WPM
1	02/06/2026	24	19	20/06/2026	26
2	03/06/2026	24	20	21/06/2026	27
3	04/06/2026	24	21	22/06/2026	27
4	05/06/2026	22	22	23/06/2026	26
5	06/06/2026	23	23	24/06/2026	26
6	07/06/2026	26	24	25/06/2026	25
7	08/06/2026	30	25	26/06/2026	26
8	09/06/2026	30	26	27/06/2026	25
9	10/06/2026	28	27	28/06/2026	27
10	11/06/2026	23	28	29/06/2026	27
11	12/06/2026	26	29	30/06/2026	30
12	13/06/2026	27	30	01/07/2026	30
13	14/06/2026	27	31	02/07/2026	29
14	15/06/2026	26	32	03/07/2026	30
15	16/06/2026	26	33	04/07/2026	27
16	17/06/2026	25	34	05/07/2026	29
17	18/06/2026	26	35	06/07/2026	29
18	19/06/2026	25	36	07/07/2026	30

6.3 Typing Progress Graph

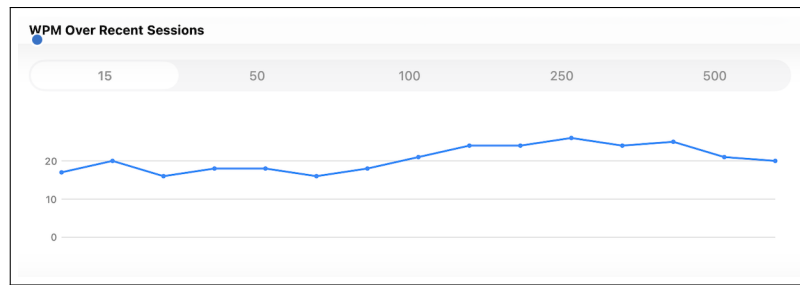


Figure 6.1: Typing Progress Graph

6.4 Observation

- Daily typing practice improved typing speed.
- Typing speed gradually increased from 22–24 WPM to approximately 30 WPM.
- Accuracy and consistency also improved during the internship.

6.5 Conclusion

Regular typing practice significantly improved typing speed, accuracy, and keyboard familiarity, which are essential skills for technical documentation and programming.

Chapter 7

HTML with MySQL Database

7.1 Aim

To develop a simple web application using HTML and store user information in an SQL database.

7.2 Objectives

1. Design a user input form using HTML.
2. Store the entered data in a MySQL database.
3. Retrieve and display records from the database.

7.3 Software Required

1. Notepad++
2. XAMPP Server
3. MySQL Database
4. PHP
5. Web Browser

7.4 Hardware Requirements

1. Desktop or Laptop Computer
2. Minimum 4 GB RAM
3. 500 MB Free Disk Space

7.5 Introduction

HTML (HyperText Markup Language) is used to create web pages and collect user information through forms. Since HTML cannot communicate directly with a database, a server-side scripting language such as PHP is used to process the submitted form data and interact with a MySQL database.

This project demonstrates a simple student registration system in which user details entered through an HTML form are stored in a MySQL database.

7.6 Theory

HTML

HTML provides the basic structure of a web page and allows developers to create forms, buttons, labels, and text fields for user interaction.

SQL

Structured Query Language (SQL) is used to create, manage, update, and retrieve information from relational databases.

PHP

PHP acts as an intermediary between HTML and MySQL. It receives the submitted form data and executes SQL queries to store or retrieve information.

7.7 Flow of the System

1. User opens the registration page.
2. User enters personal details.
3. Form data is submitted to a PHP script.
4. PHP establishes a connection with MySQL.
5. SQL query inserts the data into the database.
6. A success message is displayed to the user.

7.8 SQL Database

student_name	phy_marks	chem_marks	maths_marks	english_marks	comp_marks	total_marks	total
Rahul Kumar	85	78	92	80	88	500	
Ranish Kumar	85	78	92	80	88	500	
Karuna sharma	85	78	92	80	88	500	
Priya Sharma	90	82	95	88	91	500	
Viya Sharma	90	82	95	88	91	500	
Amit Verma	75	80	85	70	78	500	
Vinay	85	67	59	46	72	500	
Payal	30	79	99	47	90	500	
Kavita Sharma	1	32	9	7	49	500	

Figure 7.1: Student Marks Database Table

7.9 HTML Source Code

The complete HTML source code for this project is available in the GitHub repository linked in the footer.

7.10 Advantages

- Easy data management.
- Reduces paperwork.
- Fast data retrieval.
- Improves efficiency.
- Better data accuracy.

7.11 Applications

- Student Management Systems
- College Record Systems
- Employee Management
- Library Management Systems
- Hospital Registration Systems

7.12 Output

The user enters details into the HTML form. The information is transferred to a PHP script and stored successfully in the MySQL database.

7.13 Conclusion

This project demonstrated how HTML can be used to collect user information and how PHP and MySQL work together to store and retrieve data efficiently. The experiment provided practical knowledge of front-end and back-end web development.

Chapter 8

Dashboard Button Interface

8.1 Aim

To study the operation of button controls in a Battery Management System (BMS) graphical user interface.

8.2 Objective

To understand how button controls interact with software components for monitoring and controlling battery parameters.

8.3 Theory

A Button Control is a Graphical User Interface (GUI) component that executes an assigned action whenever it is clicked.

In Battery Management Systems, button controls are commonly used to

- Display battery information.
- Select battery cells.
- Monitor voltage.
- View temperature.
- Display charging status.

8.4 Software Required

1. HTML
2. CSS
3. JavaScript
4. Visual Studio Code
5. Web Browser

8.5 Procedure

1. Open Visual Studio Code.
2. Create a new HTML file.
3. Design the interface using HTML.
4. Style the interface using CSS.
5. Add JavaScript for button click events.
6. Run the application in a web browser.
7. Observe the displayed battery information.

8.6 Program Code

The complete program code for this experiment is available in the GitHub repository linked in the footer.

8.7 Screenshot

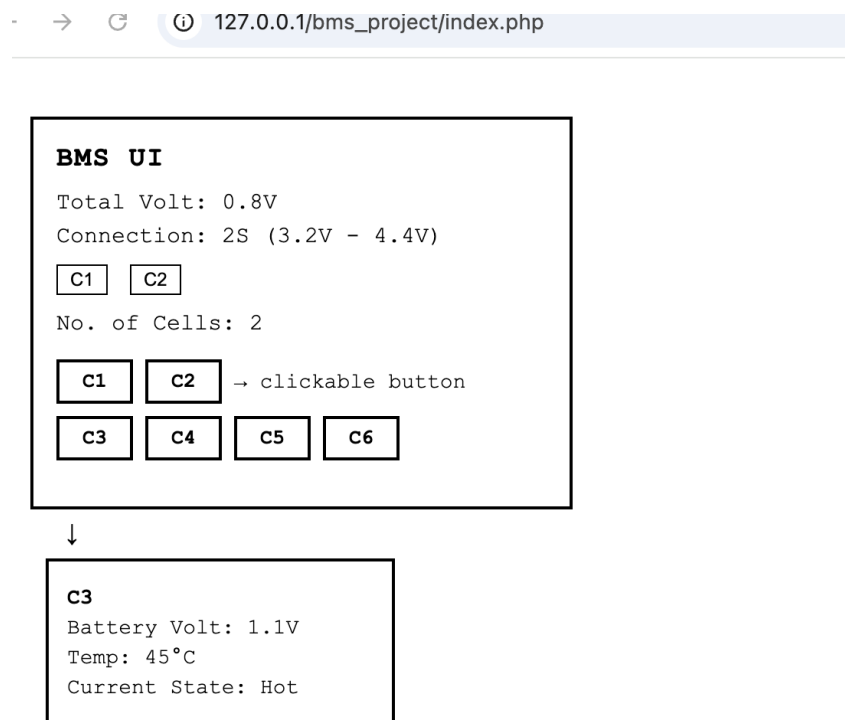


Figure 8.1: BMS Button User Interface

8.8 Observations

- Button controls responded correctly when clicked.

-
- The selected battery cell information was displayed successfully.
 - Voltage, temperature, and status were updated dynamically.
 - The graphical interface was simple and easy to operate.

8.9 Result

The BMS button interface was successfully designed and implemented. Each button displayed the corresponding battery-cell information correctly.

8.10 Conclusion

The experiment demonstrated the implementation of an interactive Battery Management System (BMS) user interface using HTML, CSS, and JavaScript. Button controls effectively displayed battery information, making the interface simple, responsive, and user-friendly.

Bibliography

- [1] Arduino, *Arduino Uno Rev3*, Arduino Documentation. Available: <https://docs.arduino.cc/hardware/uno-rev3/> Accessed: May 26, 2026.
- [2] Arduino, *Arduino IDE Documentation*. Available: <https://docs.arduino.cc/software/ide/> Accessed: May 28, 2026.
- [3] Arduino, *Seven Segment Display Tutorials and Examples*. Available: <https://docs.arduino.cc/> Accessed: June 01, 2026.
- [4] Arduino, *LED Matrix Display Documentation*. Available: <https://docs.arduino.cc/> Accessed: June 05, 2026.
- [5] Texas Instruments, *NE555 Precision Timer Datasheet*. Available: <https://www.ti.com/product/NE555> Accessed: June 20, 2026.
- [6] Texas Instruments, *LM741 Operational Amplifier Datasheet*. Available: <https://www.ti.com/product/LM741> Accessed: June 28, 2026.
- [7] Analog Devices, *Operational Amplifier Technical Articles*. Available: <https://www.analog.com/en/resources/technical-articles.html> Accessed: June 30, 2026.
- [8] Oracle Corporation, *MySQL Documentation*. Available: <https://dev.mysql.com/doc/> Accessed: July 03, 2026.
- [9] GeeksforGeeks, *PHP MySQL CRUD Operations*. Available: <https://www.geeksforgeeks.org/php-mysql-crud-operations/> Accessed: July 05, 2026.
- [10] GeeksforGeeks, *HTML Button Tag*. Available: <https://www.geeksforgeeks.org/html-button-tag/> Accessed: July 08, 2026.
- [11] MDN Web Docs, *JavaScript Guide*. Available: <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide> Accessed: July 09, 2026.
- [12] GITHUB REPOSITORY, *Source Code Repository for Internship Projects*. Available: https://github.com/kittusharma2553-design/2026_GIPEDI_KARUNA_SHARMA.git Accessed: July 10, 2026.